

GTX 1050 Mobile

Passthrough Optimus su Proxmox

Relazione tecnica: diagnosi, recupero della VBIOS OEM, SSDT ACPI dinamica, switch idempotente tra VM Linux, Secure Boot e benchmark.

Componente	Esito
Portatile	HP Pavilion Laptop 15-cs1xxx, scheda madre HP 856A
Host	Proxmox VE 9.1.5, kernel 6.17.9, IOMMU e VFIO attivi
GPU	NVIDIA GP107M GTX 1050 Mobile (Pascal), PCI ID 10de:1c8d
Firmware	VBIOS OEM HP inclusa nel repository privato, 169472 byte, 86.07.5F.00.2C
Ubuntu	Driver NVIDIA 580.173.02, 4096 MiB, nvidia-smi valido
Kali	SeaBIOS/Q35; switch reale Ubuntu -> Kali -> Ubuntu riuscito
Switch VM	Cleanup, discovery PCI/ACPI e SSDT riusciti in entrambe le direzioni
Prova rendering	Wayland/Xwayland: renderer GTX 1050; glxgears circa 60 FPS con VSync RDP

Repository: proxmox-gtx1050-mobile-passthrough

1. Problema e criterio di successo

Una GPU mobile Optimus non è una GPU desktop isolata: nel Pavilion 15-cs1xxx il display fisico resta normalmente collegato alla iGPU e il driver NVIDIA può richiedere la propria VBIOS attraverso ACPI. hostpci assegna correttamente il dispositivo PCI, ma non aggiunge automaticamente un metodo ACPI _ROM: per questo il passthrough standard con una ROM PCI non era sufficiente.

La GTX 1050 Mobile di questo caso usa GP107M, una GPU Pascal laptop con 640 CUDA core nella configurazione GTX 1050 e 4 GiB verificati nel guest. In Optimus la NVIDIA è render-only: può accelerare CUDA/OpenGL/Vulkan, ma non ha necessariamente un CRTC o un connettore display assegnabile. La procedura è quindi mirata a NVIDIA mobile/Optimus e richiede una VBIOS OEM coerente; non è una soluzione universale per ogni GPU laptop.

Il criterio di successo non è solo vedere una riga in lspci. La GPU deve essere assegnabile a una VM, il driver proprietario deve caricare, nvidia-smi deve riportare nome, driver e memoria, e un processo grafico deve usare realmente la GPU.

Questa relazione distingue sempre [NODO] e [VM]. Il nodo Proxmox e il laptop fisico: possiede GPU, IOMMU e comandi qm/lspci/gpu-vm-switch. Ubuntu e Kali sono VM: qui vivono driver NVIDIA, nvidia-smi e MOK. noVNC e la console grafica Proxmox necessaria per schermate prima del kernel, come MOK Manager.

2. Termini fondamentali

Host / guest / VMID	Host: computer fisico Proxmox. Guest: computer virtuale. VMID: numero Proxmox della VM, qui Ubuntu 1001 e Kali 1000.
BDF	Indirizzo PCI dominio:bus:device.funzione. La GPU host è 0000:02:00.0.
VBIOS / ROM	Firmware della GPU; in questo caso il file .rom contiene la VBIOS OEM.
IOMMU e VFIO	Isolano DMA e assegnano il dispositivo reale a QEMU invece che al driver host.
Optimus muxless	La iGPU guida il pannello; NVIDIA è render-only e può dipendere dai metodi ACPI della piattaforma.
ACPI	Descrizione firmware dell'hardware e dei metodi richiamati dal sistema operativo.
ASL / AML / SSDT	ASL è testo, AML è bytecode compilato, SSDT è tabella ACPI aggiuntiva.
_ROM	Metodo ACPI che fornisce una porzione della ROM al driver, dati offset e dimensione.
fw_cfg	Canale QEMU per fornire piccoli blob al guest; qui trasporta la VBIOS.
MOK	Machine Owner Key: certificato da registrare prima del kernel per fidare moduli DKMS con Secure Boot.
OVMF / Q35 / QGA	UEFI QEMU, chipset PCIe virtuale e Guest Agent usato per discovery e verifica nel guest.
nvidia-smi / nvtop / glxgears	Stato driver/VRAM, monitor GPU e rendering OpenGL con FPS. Usati insieme, non sono la stessa prova.
dry-run / self-test	Simulazione senza modifiche e compilazione/disassemblaggio AML di prova. Non sono benchmark del driver guest.

2b. Prove reali: strumenti, risultati e limiti

Ogni prova è riportata con il suo strumento, per non presentare una semplice verifica di configurazione come benchmark o compatibilità universale.

```
# [NODO] prerequisiti osservati
cat /proc/cmdline
test -d /sys/kernel/iommu_groups
lspci -nnk -s 02:00.0
gpu-vm-switch --self-test

# [VM Ubuntu] driver osservato
nvidia-smi --query-gpu=name,driver_version,memory.total --format=csv,noheader
```

- Nodo: flag IOMMU e gruppi presenti; lspci ha mostrato Kernel driver in use: vfio-pci.
- Generatore ACPI: gpu-vm-switch --self-test ha concluso self-test: ok. Prova AML, non il driver guest.
- Trasferimento reale: Ubuntu 1001 OVMF/Q35 -> Kali 1000 SeaBIOS/Q35 -> Ubuntu 1001; cleanup, discovery e SSDT sono riusciti in entrambe le direzioni.
- Ubuntu: nvidia-smi ha restituito NVIDIA GeForce GTX 1050, driver 580.173.02, 4096 MiB.
- Rendering: prima il display diagnostico :2 ha mostrato GLX NVIDIA; dopo il fix KMS il desktop Wayland/Xwayland restituisce NVIDIA GTX 1050. I circa 60 FPS di glxgears in RDP sono VSync, non benchmark di gioco.
- Secure Boot off: il codice/prompt sono stati esaminati ma la sostituzione EFI non è stata eseguita sulla Ubuntu principale; nessuna prova runtime viene rivendicata.

2c. Claim laptop: cosa si applica davvero

- Optimus muxless: coerente con il caso. La dGPU può rendere senza guidare il pannello; non è stato mappato il cablaggio di ogni porta fisica del Pavilion.
- VBIOS: il file HP è integro (55 aa, PCIR, 10de:1c8d, 169472 byte). Qui non è stato tagliato alcun header UEFI: ROM BAR sola falliva, fw_cfg + SSDT _ROM funziona.
- FLR/reset: non è stato provato che manchi. Il test Ubuntu -> Kali -> Ubuntu senza reboot host dimostra che il reboot Proxmox non è obbligatorio in questo flusso; reset_method resta un controllo sysfs da eseguire senza scrivere reset.
- D3cold/_DSM: sono rischi laptop possibili ma la SSDT risolve _ROM, non emula metodi energetici proprietari. Non sono stati attribuiti come causa del guasto.
- IOMMU/ACS: VFIO richiede valutare il gruppo; lo script non usa ACS override. Code 43 è Windows-specifico e non è stato testato dai guest Linux.
- GVT-g, SR-IOV, Looking Glass e Sunshine/Moonlight sono alternative o trasporto display: non sostituiscono VFIO/VBIOS/ACPI e non sono stati installati qui.

3. Perché la soluzione ACPI era necessaria

Il solo romfile espone una ROM nella configurazione PCI virtuale. Il driver della GTX 1050 Mobile, nel percorso Optimus, cerca invece _ROM nel device ACPI. Per questo rombar=1 mostrava byte ROM ma non risolveva l'inizializzazione del driver. VBIOS e file .rom sono qui lo stesso firmware; rombar è invece una finestra PCI virtuale, non il metodo ACPI.

La soluzione passa lo stesso file VBIOS OEM mediante fw_cfg. Una SSDT in AML si aggancia al device della GPU, legge il file una volta, lo mantiene in un buffer e implementa _ROM restituendo il segmento richiesto. rombar=0 resta intenzionale.

```
gtx1050_hp_native.rom sul nodo
|
v
```

```
QEMU fw_cfg -> SSDT AML (buffer FWBI) -> _ROM(offset,length) -> driver NVIDIA
0000:02:00.0
|   |   |   |-- funzione 0: GPU
|   |   |   +----- dispositivo 00
|   +----- bus 02
+----- dominio PCI 0000
```

4. PCI -> ACPI senza valori fissi

Il guest restituisce la catena PCI con lspci -PP. Ogni hop viene convertito nel nome ACPI Sxx: valore = slot * 8 + funzione.

```
00:1c.0/01:00.0
1c.0 -> 0x1c * 8 + 0 = 0xe0 -> SE0
00.0 -> 0x00 * 8 + 0 = 0x00 -> S00
Scope finale: \_SB.PCI0.SE0.S00
```

ASL e il testo generato; iasl lo compila in AML, il bytecode che QEMU carica. External dichiara il device già presente nella DSDT, Scope lo apre, RINT legge fw_cfg nel buffer FWBI e _ROM restituisce Mid(FWBI, offset, length). Lo script avvia brevemente la VM, scopre questa topologia reale e genera la SSDT specifica della VM. Così il metodo può adattarsi a un'altra NVIDIA mobile, se vengono indicati BDF host e VBIOS OEM corretti.

4b. SSDT: lettura guidata

```
External (\_SB.PCI0.SE0.S00, DeviceObj) # device esistente
Scope (\_SB.PCI0.SE0.S00) # aggiungi metodi alla GPU
Name (FWIT, 0) / Name (FWBI, Buffer(){} ) # flag + buffer VBIOS
OperationRegion (... 0x510, 2) # porte QEMU fw_cfg
FISL (...) # trova nome e dimensione blob
RINT () # carica una volta FWBI
_ROM (offset, length) -> Mid(FWBI,...) # risposta al driver
```

RWRD, RDWD e RBUF sono lettori rispettivamente a 16 bit, 32 bit e buffer; Serialized evita letture concorrenti. Il walkthrough del repository spiega ogni blocco e anche il limite a 4 KiB per richiesta _ROM.

5. Switch GPU idempotente

- Prima del primo switch: --prepare-host installa acpica-tools/pciutils, installa la ROM OEM se diversa, aggiunge solo flag IOMMU/VFIO mancanti e aggiorna initramfs; il reboot è sempre esplicito.
- Menu numerato oppure --vm VMID; preflight di Q35, firmware, Guest Agent, BDF e ROM. Secure Boot viene letto nel guest prima del cleanup.
- Ramo idempotente: se una sola VM ha già hostpci, SSDT, fw_cfg e nvidia-smi funzionante, il tool non modifica né riavvia.
- Ricerca proprietario e cleanup mirato: soltanto hostpci della GPU, romfile/rombar collegati, argomenti SSDT/fw_cfg, CPU hidden e SSDT generata dal tool.
- Assegnazione alla destinazione, boot temporaneo con lspci -PP, conversione PCI->ACPI, ASL->AML e boot finale con -acpitable e -fw_cfg.
- Driver della distro, riavvio se serve, nvidia-smi e benchmark. Il trap finale riaccende la sorgente prima attiva senza GPU, anche dopo un errore.
- Nessuna conversione forzata di BIOS, OVMF, SeaBIOS, Q35 o vga: questi restano proprietà della VM.
- Se hostpci, SSDT/fw_cfg e nvidia-smi sono già corretti nella VM richiesta, non opera né riavvia; se resta solo MOK, non ricostruisce lo switch.
- Test reale completato: Ubuntu 1001 OVMF/Q35 -> Kali 1000 SeaBIOS/Q35 -> Ubuntu 1001; cleanup e SSDT sono stati rigenerati in entrambe le direzioni.

Comandi principali

```
gpu-vm-switch --prepare-host --rom-source ./firmware/gtx1050_hp_native.rom --yes
gpu-vm-switch --prepare-host --rom-source ./firmware/gtx1050_hp_native.rom --reboot --yes
gpu-vm-switch
gpu-vm-switch --vm 1001 --yes
gpu-vm-switch --vm 1001 --dry-run --yes
gpu-vm-switch --gpu 0000:03:00 --rom /usr/share/kvm/oem.rom --vm 123 --yes
gpu-vm-switch --vm 123 --skip-drivers --yes
```

Driver automatici: Ubuntu, Debian, Kali, Arch, Fedora, RHEL, Rocky e AlmaLinux. Per Debian la sorgente contrib non-free non-free-firmware è aggiunta solo se non è già presente; la logica gestisce anche il formato Deb822. Il BIOS del laptop resta l'unico prerequisito non automatizzabile: VT-d/AMD-Vi deve essere attivo prima del boot Proxmox.

5b. Procedura riproducibile e criteri di prova

```
# nodo: prima di applicare
sha256sum firmware/gtx1050_hp_native.rom
lspci -nnk -s 0000:02:00.0
gpu-vm-switch --prepare-host --rom-source ./firmware/gtx1050_hp_native.rom --dry-run --yes

# nodo: applicazione e controllo dopo reboot
gpu-vm-switch --prepare-host --rom-source ./firmware/gtx1050_hp_native.rom --yes
cat /proc/cmdline; lspci -nnk -s 0000:02:00.0; gpu-vm-switch --self-test

# switch: simulazione, applicazione, prova driver
gpu-vm-switch --vm 1001 --dry-run --yes
gpu-vm-switch --vm 1001 --yes
qm guest exec 1001 -- /usr/bin/nvidia-smi --query-gpu=name,driver_version,memory.total --format=csv,noheader
```

La sequenza è deliberatamente divisa in inventario, dry-run, applicazione e osservazione. Il risultato positivo richiede: hash ROM atteso, IOMMU e vfio-pci dopo reboot, self-test AML riuscito, hostpci con args SSDT/fw_cfg e nvidia-smi con exitcode 0. nvidia-glxgears e nvtop provano poi il rendering. Il runbook del repository aggiunge comandi esatti per QEMU Guest Agent, Secure Boot/MOK, prova Ubuntu -> Kali -> Ubuntu e condizioni in cui fermarsi; il validatore documentale non viene spacciato per prova hardware.

6. Secure Boot senza automazione fittizia

MOK non è NVIDIA e non è una password Linux. Secure Boot è la politica UEFI della VM; DKMS può compilare nvidia.ko localmente e firmarlo con una chiave nuova; MOK (Machine Owner Key) è il certificato pubblico autorizzato dal proprietario; shim porta quella autorizzazione nella catena Linux.

Se il pacchetto contiene un modulo già firmato da una chiave fidata, MOK non compare. Il caso MOK è: driver installato -> DKMS compila e firma -> il kernel non conosce ancora il certificato pubblico -> il modulo è rifiutato. La password temporanea di mokutil protegge solo la richiesta pendente fino al reboot; non firma il modulo e non è la password dell'account.

```
script: legge Secure Boot prima del cleanup -> installa driver se manca -> prova nvidia-smi
-> se fallisce con Secure Boot: stampa istruzioni, ma non esegue mokutil --import
-> utente: mokutil --import certificato.der -> reboot
-> UEFI avvia shim; MOK Manager prima del kernel
-> Enroll MOK -> Continue -> Yes + password temporanea
-> reboot: shim rende la chiave disponibile, modulo ammesso, nvidia-smi
-> gpu-vm-switch --mok-manual: sola verifica idempotente
```

Lo script legge lo stato reale nel guest con mokutil oppure con la variabile EFI SecureBoot prima del cleanup e dopo QEMU Guest Agent. Nel menu interattivo propone Secure Boot off; con --yes non lo cambia mai senza --disable-secure-boot. Con --mok-manual conserva Secure Boot: soltanto se il controllo finale nvidia-smi fallisce, lascia GPU/SSDT/fw_cfg invariati, mostra chiavi pendenti e cerca certificati DKMS per noVNC; l'import e la schermata MOK restano manuali.

```
gpu-vm-switch --vm 123 --mok-manual
gpu-vm-switch --vm 123 --disable-secure-boot --yes
```

MOK Manager è prima del kernel: SSH, rete e QEMU Guest Agent non possono premere tasti; appare solo se una richiesta è pendente. L'alternativa Secure Boot off è permanente e solo OVMF/efidisk0 4m. Prima crea EFI/BOOT/BOOTX64.EFI, copia variabili e metadata in /usr/share/kvm/optimus-gpu-switch/efi-backups/ e sostituisce efidisk0 con un template senza chiavi; l'originale rimane unused. Il rollback resta attivo fino al primo boot con Guest Agent. Questo evita MOK ma abbassa la protezione della catena di boot e non è stato eseguito sulla Ubuntu principale: usare snapshot e noVNC.

6 - Accesso RDP Wayland della VM Ubuntu

Il primo server RDP era xrdp/xorgxrdp: autenticava ma apriva una sessione X11. GNOME 50 richiedeva XDG_SESSION_TYPE=wayland e terminava la sessione, causando finestra nera/disconnessione. Il fallback Flashback X11 ha mostrato il desktop, ma non e la soluzione Wayland. Il 2026-08-27 sono stati rimossi Flashback, xrdp, xorgxrdp, file di sessione e processi X11 orfani. L'autoremove non e stato eseguito: il driver 580 attivo e stato poi marcato manuale e una simulazione non propone pacchetti NVIDIA.

E stato configurato GNOME Remote Desktop 50.2 in modalita system (Remote Login):
gnome-remote-desktop-daemon e attivo sulla porta 3389. Il client e sempre mstsc.exe Windows: il profilo RDSTLS UTF-16 fornisce use redirection server name:i:1 e audiomode:i:0 per riprodurre l'audio remoto sul PC. L'audio playback e stato ascoltato e confermato manualmente dall'utente. Il 2026-08-27 e stata trovata la causa della differenza tra file e GUI: Documents\Default.rdp, il profilo nascosto della GUI mstsc, aveva use redirection server name:i:0. Dopo il redirect GNOME il client sceglieva NLA e il server riportava SAM database/SEC_E_NO_CREDENTIALS, non un guasto GPU. Il valore e stato corretto e verificato a :i:1; quindi anche digitando l'IP nella app preinstallata eredita RDSTLS. Il server emette davvero Sending server redirection e avvia il greeter/handover. Nello stesso giorno e stato creato lo snapshot Proxmox pre-rdp-handover-backport-20260827 e installato il backport locale gnome-settings-daemon 50.0-1ubuntu1+rdphandover1: una guardia impedisce a gsd-sharing di spegnere il servizio di hand-over mentre il daemon RDP di sistema e vivo. Il test visivo diretto mstsc greeter->desktop dopo questa correzione resta da ripetere. Nessun PPA e stato installato.

7. Benchmark e prova di rendering

glmark2 da SSH ha fallito con Could not initialize canvas: e previsto, perche glmark2 X11 necessita un display. Il DRM della GPU mobile render-only non offre un CRTC utile al test. E stato quindi creato Xorg NVIDIA headless su :2 e il wrapper nvidia-glxgears: e un display diagnostico, non il desktop Wayland.

```
# Nel terminale della sessione RDP/Wayland
glxinfo -B | grep -E 'OpenGL vendor|OpenGL renderer'
# NVIDIA Corporation / NVIDIA GeForce GTX 1050/PCIe/SSE2
glxgears # circa 60 FPS: sincronizzato a VSync RDP
__GL_SYNC_TO_VBLANK=0 glxgears # solo contatore non sincronizzato
watch -n 1 nvidia-smi
nvidia-smi
```

La causa del software rendering era nvidia_drm modeset=0 in due vecchi file locali, non VFIO o la VBIOS. Dopo modeset=1, initramfs e reboot, GNOME seleziona nvidia-drm primario e Xwayland usa NVIDIA. htop non visualizza i contatori NVIDIA perche legge CPU e RAM; nvidia-smi e nvidia-smi interrogano invece driver e GPU.



Figura 1 - precedente prova sul display diagnostico Xorg :2: prova renderer, non benchmark universale.

7b. Omarchy: desktop headless GTX e Moonlight

Il 28 agosto 2026 è stato verificato un percorso distinto dal precedente desktop VirtIO: Hyprland usa `AQ_DRM_DEVICES=/dev/dri/gtx1050` e `AQ_NO_KMS_REQUIREMENT=1`, crea l'uscita Wayland headless omarchy-gtx e Sunshine la cattura a 1920x1200/60, scala 1. Il campione Moonlight HEVC ha mostrato 60.19 FPS, host processing latency 7.6/8.8/7.9 ms e 0% frame persi; `nvidia-smi pmon` ha mostrato Hyprland come processo grafico della GTX e Sunshine con `enc=28`. Il mismatch precedente era tra GBM 1920x1080 e il client 1920x1200, quindi la sorgente viene ora fissata e verificata a 1920x1200.

```
GTX -> Hyprland -> omarchy-gtx (1920x1200@60)
        -> Sunshine GBM/Wayland -> h264_nvenc o hevc_nvenc -> Moonlight
prova: Hyprland nella tabella NVIDIA; Sunshine enc>0 durante lo stream
```

L'output QEMU VirtIO rimane nel file Proxmox soltanto come console noVNC di recupero: non è selezionato dal compositor. Non è stato quindi eseguito `vga:none` senza un'ulteriore prova utente; questa scelta preserva una strada di rollback. Il tentativo di forzare una EDID sul connettore NVIDIA disconnesso aveva lasciato guest SSH e QEMU Guest Agent non raggiungibili ed è stato rimosso, non mantenuto.

Il build stabile precedente, compatibile con il driver Pascal 580, aveva `SUNSHINE_ENABLE_CUDA=OFF` e forzava frame NV12 in RAM per evitare un errore di scala: NVENC restava hardware ma la CPU non era zero. Nel test Sunshine era circa al 51% di una CPU logica, mentre Hyprland circa al 3-4%; `htop` può mostrare i thread separati e non devono essere sommati. Il pacchetto Sunshine Arch ufficiale è stato provato e subito annullato: `h264_nvenc` ha fallito per reference frames non supportati sulla GTX 1050 ed è ricaduto a `libx264` software.

Il 28 agosto 2026 è stato compilato in un percorso separato un canary Sunshine con CUDA 12.8 e GCC 14 isolati: compila `cuda.cu` con `--generate-code` per `sm_61`, ha `SUNSHINE_ENABLE_CUDA=ON` ed è installato in `/opt/sunshine-cuda12`. La drop-in `29-cuda12-canary.conf` lo rende attivo soltanto se Sunshine trova `h264_nvenc`; può essere ritirata in modo idempotente da `omarchy-sunshine-cuda12-canary rollback`. Il probe ha inoltre trovato `hevc_nvenc`: Sunshine ora rileva automaticamente HEVC e lascia H.264 come fallback, mentre AV1 resta disabilitato perché Pascal non lo codifica in hardware. CUDA non attiva NVENC, ma può mantenere in VRAM la superficie catturata prima dell'encoder e ridurre copie CPU/RAM. Il campione runtime prova HEVC/NVENC; non dichiara zero-copy o una riduzione CPU finché non viene confrontato con un campione equivalente. Il preset P3 è mantenuto: 7.9 ms host e zero drop non richiedono P2/P1. CUDA 13 non può compilare offline per Pascal compute capability 6.1.

8. Estrazione della VBIOS OEM

Il punto di partenza è il pacchetto driver/firmware ufficiale HP per il Pavilion 15-cs1xxx, estratto localmente fino al payload, per esempio `084C0.bin`. Lo script Python non scarica nulla: cerca la signature PCI option ROM 55 aa, verifica PCIR, vendor NVIDIA e device 1c8d. Scansiona il payload RAW e stream LZMA, quindi estrae tutte le immagini della ROM fino al flag finale.

```
python3 scripts/extract_gtx1050_rom.py /tmp/084C0.bin \
--device-id 1c8d \
--output /usr/share/kvm/gtx1050_hp_native.rom
```

Per rendere riproducibile questo laboratorio privato, la ROM estratta è inclusa come `firmware/gtx1050_hp_native.rom`: 169472 byte, VBIOS 86.07.5F.00.2C, SHA-256 33abd3bc3f658b0536da0617a76076b56e5af124701271211d6d127cda22c322. È firmware OEM, non universale: verificare licenza e usare `--force` solo dopo confronto di vendor, device ID e origine del payload.

9. Tentativi che non hanno risolto il problema

Tentativo	Perché falliva	Correzione
Solo romfile / rombar	Il driver Optimus non leggeva la ROM dalla sola finestra PCI.	SSDT_ROM + fw_cfg.

ACPI scritto a mano	Cambiando bridge cambia lo scope del device.	Discovery lspci -PP e conversione Sxx.
glmark2 via SSH	Mancava un canvas X11.	Xorg NVIDIA :2 e glxgears.
Wayland con llvmpipe	Due override locali disabilitavano nvidia_drm KMS.	modetest=1, initramfs, reboot e glxinfo NVIDIA.
Autoremove NVIDIA	Pacchetti attivi potevano essere automatici.	apt-mark manual e simulazione pulita; script lo fa dopo install.
Prima installazione Kali	Headers/DKMS non pronti.	Headers, build tools, DKMS e riavvio.
Repo Debian riscritta	Automazione non idempotente.	Verifica componenti prima di aggiungere.

10. Ripetere il procedimento

Checklist: verificare IOMMU/VFIO e audio; estrarre la VBIOS OEM e controllare BDF/PCI ID; configurare Q35 e Guest Agent; usare gpu-vm-switch con --gpu e --rom; testare il renderer con nvidia-glxgears su display NVIDIA headless; scegliere consapevolmente MOK oppure --disable-secure-boot per OVMF.

Riferimenti

QEMU fw_cfg: https://qemu-project.gitlab.io/qemu/specs/fw_cfg.html

ACPI: <https://uefi.org/acpi/specs>

Proxmox VE Administration Guide: <https://pve.proxmox.com/pve-docs/pve-admin-guide.html>

NVIDIA driver kernel modules: <https://docs.nvidia.com/datacenter/tesla/driver-installation-guide/kernel-modules.html>

NVIDIA GTX 1050 laptop / Pascal: <https://www.nvidia.com/en-us/geforce/news/nvidia-geforce-gtx-1050-laptops/>

Linux kernel VFIO: <https://docs.kernel.org/driver-api/vfio.html>

Linux PCI reset sysfs ABI: <https://docs.kernel.org/6.10/admin-guide/abi-testing.html>

NVIDIA Optimus Linux: https://download.nvidia.com/XFree86/Linux-x86_64/455.28/README/optimus.html

NVIDIA PRIME Render Offload: https://download.nvidia.com/XFree86/Linux-x86_64/575.64/README/primerenderoffload.html

Intel i915 / GVT-g: <https://docs.kernel.org/next/gpu/i915.html>

ACPICA / iasl: <https://acpica.org/>

Hyprland Virtual GPU: <https://wiki.hypr.land/Configuring/Advanced-and-Cool/Virtual-GPU/>

Hyprland hyprctl: <https://wiki.hypr.land/Configuring/Advanced-and-Cool/Using-hyprctl/>

NVIDIA CUDA 13 release notes: <https://docs.nvidia.com/cuda/archive/13.0.0/cuda-toolkit-release-notes/index.html>

NVIDIA CUDA architecture / driver matrix: <https://docs.nvidia.com/datacenter/tesla/drivers/cuda-toolkit-driver-and-architecture-matrix.html>

Sunshine build documentation: <https://github.com/LizardByte/Sunshine/blob/master/docs/building.md>

GNOME Remote Login: <https://teams.pages.gitlab.gnome.org/Websites/help.gnome.org/gnome-help/remote-login.html>